Structure-Aware Delta Debugging with Geometric–Information Weights

ANONYMOUS AUTHOR(S)

Delta debugging is a fundamental technique for automatically minimizing failure-inducing inputs. ProbDD improves ddmin via probability-guided search, and Weighted Delta Debugging (WDD) further incorporates token-based weighting to mitigate size disparities. However, token counts measure textual volume rather than structural complexity, potentially misrepresenting reduction difficulty.

We propose Structure-Aware Delta Debugging (SADD), which models structural complexity using geometric and information-theoretic properties of the syntax tree. SADD defines a unified weight function that integrates geometric volume, decision uniformity, and effective branching complexity. We instantiate this model in two variants: SA_{ddmin}, which performs structure-aware partitioning within ddmin, and SA_{ProbDD}, which injects structural weights into ProbDD's gain function, leaving the underlying search control unchanged.

We evaluate SADD against ddmin, ProbDD, and WDD (instantiated as W_{ddmin} and W_{ProbDD}) within two state-of-the-art frameworks, HDD and Perses, on 62 real-world C and XML benchmarks. Under HDD, SADD reduces average debugging time by up to 57.12% over ddmin and 15.04% over W_{ddmin} on C, and by 30.12% over ddmin on XML; improvements over ProbDD reach 14.64% on C and 20.10% on XML. Gains are smaller under Perses or on structurally simple inputs, where token-based weighting already suffices. An ablation study shows that geometric volume forms the foundation of improvement, while information-theoretic metrics provide complementary refinements. Overall, explicitly modeling structural complexity improves delta debugging when input hierarchies exhibit meaningful structural diversity.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Delta Debugging, Program Reduction

ACM Reference Format:
Anonymous Author(s). 2026. Structure-Aware Delta Debugging with Geometric–Information Weights. In *Proceedings of the 2026 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE '26).* ACM, New York, NY, USA, 22 pages. https://doi.org/XXXXX.XXXXX

1 Introduction

Delta debugging [32] is a widely used family of algorithms designed to minimize failure-inducing programs while preserving the target failure behavior. This systematic approach to program reduction has demonstrated its versatility and effectiveness across various software engineering domains, including compiler testing [3, 5, 12, 20, 30], SMT solver testing [26, 31], fault localization [4, 33], and software debloating [7, 35].

The practical importance of delta debugging becomes evident in real-world software engineering workflows. In compiler testing, failure-inducing test cases that trigger compiler bugs are often large and complex, sometimes spanning thousands of lines. Both GCC and LLVM explicitly recommend test-case reduction in their bug-reporting guidelines [6, 13], acknowledging that minimal reproducible examples substantially ease debugging and long-term maintainability.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FSE '26, Montreal, Canada
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXX.XXXXX

Delta debugging algorithms typically treat test inputs as lists of elements, where each element corresponds to a program fragment (e.g., tokens, lines, or tree nodes), and systematically partition these lists to identify and remove failure-irrelevant elements. The foundational DDMIN algorithm [34] applies a divide-and-conquer strategy with uniform partitioning (based on the number of elements per partition). In practice, delta debugging algorithms are frequently applied to tree-structured representations rather than flat element lists, as this approach typically yields superior minimization performance. Two prominent examples are HDD [15], which centers on DDMIN by applying it hierarchically at every tree level, and PERSES [21], where DDMIN plays only a peripheral role, applied to individual elements after grammar-based structural transformations during Abstract Syntax Tree (AST)-based reduction. This tree-based paradigm has proven highly effective, spawning numerous specialized tools and techniques [23, 28, 29, 36–39]. A distinct line of research has primarily focused on leveraging historical feedback to reduce redundancy in delta debugging: CHISEL [7] uses reinforcement learning to predict promising deletions, while PROBDD [24] adopts a probabilistic selection strategy based on the historical success rates of individual program elements. CDD [37] conducts the first in-depth theoretical analysis of PROBDD, revealing that its probabilistic model essentially functions as monotonically increasing counters and proposing a simplified variant that achieves equivalent performance with PROBDD.

Challenges. Existing delta debugging approaches rely on surface-level metrics that often fail to capture structural complexity, leading to inefficient partitioning and unnecessary tests. Uniform partitioning offers generality but treats structurally different elements identically. Recent work such as WDD [39] introduces token-based weighting, instantiated as W_{DDMIN} and W_{PROBDD} by extending DDMIN and PROBDD with token counts (Section 2.5). However, token counting remains a syntactic proxy and cannot distinguish elements with different structural complexity within program hierarchies. This limitation arises when elements with similar token counts play markedly different structural roles, such as deeply nested conditionals versus flat variable declarations.

Key Insight. Program elements with similar surface-level sizes can differ greatly in structure and potential impact. This suggests that, in delta debugging, removability is often more closely reflected by structural complexity than by syntactic size in tokens. Tokens are a coarse proxy: a verbose but structurally simple element (e.g., a `typedef`) may contribute many tokens yet be easy to remove, while a compact but deeply nested construct (e.g., multi-level control flow) may resist removal despite modest token count. Token count measures textual volume, whereas reduction difficulty—i.e., the likelihood that removing an element or subset preserves the failure-inducing behavior—depends more on structural entanglement within the syntax tree. By quantifying geometric, distributional, and branching properties, we obtain a more discriminative notion of "size" that better aligns partitioning and selection decisions with actual reduction difficulty, especially in structurally heterogeneous programs. While structural complexity does not guarantee removability, it provides a more discriminative heuristic signal than token count for guiding reduction decisions.

Our Approach. We present Structure-Aware Delta Debugging (SADD), which incorporates multidimensional structural complexity as a complementary factor in delta-debugging partitioning and selection. SADD uses a unified weight function combining geometric and information-theoretic properties to quantify the structural reduction difficulty of each element. While WDD replaces element count with token count, it still treats size as textual volume rather than structural complexity. SADD refines this notion by measuring structural complexity, which provides a closer proxy for reduction difficulty in practice. We instantiate it in two algorithms: SA_{DDMIN} augments DDMIN with structure-aware partitioning: instead of equal-element-count (as in DDMIN) or equal-token splits (as in W_{DDMIN}), it creates partitions balanced by structural weight, so each group reflects comparable reduction difficulty rather than comparable verbosity. This heuristically increases the likelihood that early tests isolate structurally removable regions. SA_{PROBDD} integrates structural weights into

ProbDD's probabilistic selection: instead of maximizing expected token removal as in W_{ProbDD}, it maximizes expected structural-weight reduction, biasing selection toward eliminating greater structural complexity. This shifts the objective from textual reduction to structural simplification, better reflecting meaningful reduction progress. In both cases, structural weight acts as a drop-in replacement for token-based signals (partitioning in SA_{ddmin} and gain ranking in SA_{ProbDD}), while leaving control structure and termination unchanged. Overall, SADD improves ddmin and ProbDD by making partitioning and candidate ranking structure-aware, and improves WDD by replacing token-count weights with a multidimensional structural signal. Because SADD modifies only the weighting signal while leaving the underlying search control unchanged, comparisons with WDD isolate the effect of weighting quality. These two variants are implemented in HDD [15] (where ddmin is central) and Perses [21] (which applies structural transformations before ddmin, where ddmin plays only a peripheral role) and evaluated on 62 real-world benchmarks (C as a general-purpose language and XML as a DSL).

Results. We evaluate SADD against ddmin, W_{ddmin}, W_{ProbDD}, and ProbDD on 62 real-world C and XML benchmarks using HDD and Perses. Under HDD, SADD markedly reduces average debugging time: **57.12%** (SA_{ddmin} vs. ddmin), **15.04%** (SA_{ddmin} vs. W_{ddmin}), and **14.64%** (SA_{ProbDD} vs. ProbDD) on C benchmarks, and **30.12%** (SA_{ddmin} vs. ddmin) and **20.10%** (SA_{ProbDD} vs. ProbDD) on XML benchmarks. On C benchmarks under Perses, SA_{ProbDD} achieves a **7.02%** reduction over ProbDD. In other cases—such as XML programs with simpler, flatter structures where WDD's token counting already suffices, or under Perses, which applies structural transformations before ddmin—the gains are expectedly limited or slightly negative (e.g., −7.43% for SA_{ddmin} vs. W_{ddmin} on XML programs under HDD).

An ablation study shows geometric volume provides the foundation of these improvements, while information-theoretic metrics add complementary, context-sensitive refinements. Overall, modeling structural complexity enhances delta debugging, especially when input hierarchies exhibit diversity that token counts alone cannot capture.

Contributions. This paper makes the following contributions:

- We present Structure-Aware Delta Debugging (SADD), which incorporates multi-dimensional structural complexity into delta-debugging partition decisions.
- We realize SADD in two representative delta debugging algorithms, ddmin and ProbDD, resulting in SA_{ddmin} and SA_{ProbDD}, respectively.
- We evaluate SA_{ddmin} and SA_{ProbDD} on 62 C and XML benchmarks across two delta debugging frameworks, HDD and Perses, showing that incorporating multi-dimensional structural complexity can substantially accelerate debugging, while also revealing scenarios where its benefits are limited or absent.
- We release all artifacts and code to support reproducibility and future work.

2 Preliminaries

In this section, we establish the foundations of our structure-aware delta debugging approach. We first formalize the delta debugging problem and the hierarchical tree-based program representations on which modern reducers operate (Sections 2.1 and 2.2). We then review three key partitioning strategies: the canonical ddmin with uniform partitioning (Section 2.3), its probabilistic enhancement ProbDD (Section 2.4), and the state-of-the-art WDD with token-based weighting (Section 2.5). We conclude by highlighting how these methods remain oblivious to deeper structural properties, motivating the novel complexity metrics introduced in our approach.

```
1 typedef long long llong;
2 typedef llong vllong2 __attribute__((__vector_size__(sizeof(llong))));
3 typedef llong vllong1 __attribute__((__vector_size__(sizeof(llong))));
4 test2char64(char *p) {}
5 test2short32(short *p) {}
6 test1char8(char c) {}
7 vllong1 test2llong1(llong *p) {llong c = *test1char8; vllong1 v = {c}; return v;}
8 int main() {}
```

Fig. 1. A program that triggers the real-world compiler bug GCC-71626

2.1 Delta Debugging Fundamentals

Delta debugging is a systematic approach for automatically minimizing a failure-inducing test input to a smaller subset that preserves the failure-triggering behavior. Formally, let an input l be a list of elements $[e_1, e_2, \ldots, e_n]$. A test function $\psi : \mathbb{L} \rightarrow \mathbb{B}$ evaluates a given list from the universe of possible inputs $\mathbb{L}$, where $\mathbb{B} = \{\textbf{true}, \textbf{false}\}$ represents boolean values. The function ψ returns **true** if the list preserves the property of interest (e.g., triggers a bug), and **false** otherwise.

Given an initial input l_{fail}, where $\psi(l_{fail}) = \textbf{true}$, the goal of delta debugging is to find an output list l_{min} that satisfies the following three conditions:

(1) **Subset:** l_{min} is a sub-list of l_{fail}.
(2) **Property Preservation:** The failure-inducing property is preserved, i.e., $\psi(l_{min}) = \textbf{true}$.
(3) **1-Minimality:** The output is locally minimal: removing any element from l_{min} invalidates the property. Formally, for every $e \in l_{min}$, $\psi(l_{min} \setminus \{e\})$ returns **false**.

This **1-minimality** is the widely adopted criterion for delta debugging algorithms, as obtaining a globally minimal result has been proven to be NP-complete [15, 33].

2.2 Tree-based Program Representation

Modern delta debugging algorithms typically operate on structured representations rather than flat text. Programs are decomposed into an AST representation, where nodes represent program constructs (statements, expressions, declarations) and edges capture syntactic relationships.

Consider the C program in Figure 1, which triggers compiler bug GCC-71626 [2]. Tree-based reducers like HDD [15] and Perses [21] parse this into a hierarchical structure yielding eight top-level AST nodes: three typedef declarations (lines 1-3), three simple function definitions (lines 4-6), one function definition with body (line 7), and a main function (line 8), as illustrated in Figure 2.

This representation preserves syntactic validity[1]—for example, removing an entire `typedef` declaration maintains valid C syntax, whereas deleting arbitrary character ranges may not. However, although hierarchical organization provides natural element boundaries, existing partitioning strategies treat these elements uniformly or by surface-level metrics, without leveraging their structural characteristics.

The algorithms we examine next—ddmin, ProbDD, and WDD —all operate on such tree-level element representations, recursively minimizing subtrees as needed.

2.3 The ddmin Algorithm

ddmin [34] is the foundational algorithm in this domain. It uses a divide-and-conquer strategy to reduce input size through iterative partitioning and testing in three steps:

[1] Throughout this paper, *syntactic validity* refers to grammar compliance (parseability), consistent with HDD [15] and Perses [21]. Semantic validity (e.g., successful compilation and failure reproduction) is enforced by the test oracle.

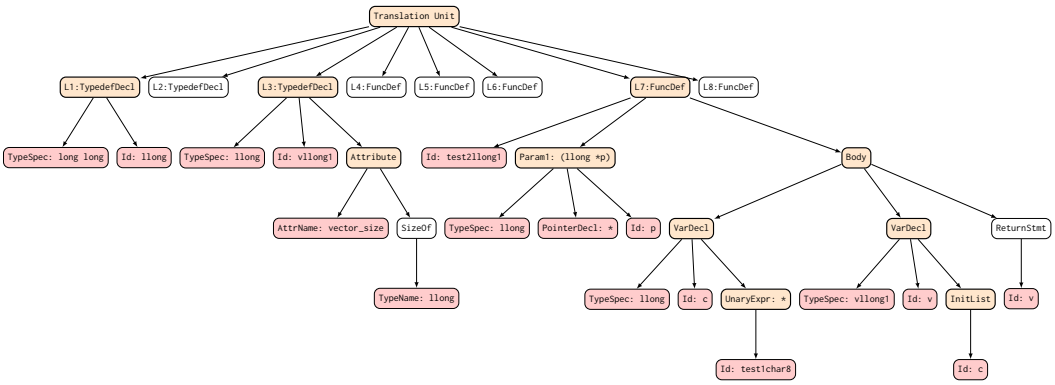

Fig. 2. An AST of the C code from Figure 1 illustrating structural complexity. For clarity, only representative nodes L1, L3, and L7 are expanded. Decision nodes have $k \geq 2$ children, while leaf nodes represent terminals. The contrast among L1 (simple declaration), L3 (declaration with attributes), and L7 (function with body) shows structural diversity beyond what token-based weighting can capture.

Inputs	l_1	l_2	l_3	l_4	l_5	l_6	l_7	l_8	l_9	l_{10}	l_{11}	l_{12}	l_{13}	l_{14}	l_{15}	l_{16}	l_{17}	l_{18}	l_{19}	l_{20}	l_{21}	l_{22}	l_{23}	l_{24}	l_{25}	l_{26}	l_{27}	l_{28}	l_{29}	l_{30}
Granularity (n)	2		4								8										7		6			5				
Initial Elements e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1
e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2
e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3
e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4
e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5
e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6
e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7
e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8
ψ	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	**T**	F	**T**	F	F	**T**	F	F	F	F	F

Fig. 3. Step-by-step trace of DDMIN and W_{DDMIN} on the motivating example in Figure 1. With uniform token distribution, both produce identical reduction traces. Each step shows a test case with statements highlighted in cyan, flagged '**T**' if they satisfy ψ and 'F' otherwise.

Step 1 (Reduce to Subset): Split l into n partitions of equal length, starting with $n = 2$. For each partition p, test whether p alone preserves the property, i.e., $\psi(p) = \textbf{true}$. If any partition succeeds, update $l \leftarrow p$ and restart Step 1 with $n = 2$; otherwise, proceed to Step 2.

Step 2 (Reduce to Complement): Test whether the complement of each partition preserves the property, i.e., $\psi(l \setminus p) = \textbf{true}$. If any complement test succeeds, remove the corresponding partition from l and restart Step 1 with $n \leftarrow n - 1$; otherwise, proceed to Step 3.

Step 3 (Granularity Adjustment): If no progress is made and partitions can be further divided (i.e., $n < |l|$), double the number of partitions ($n \leftarrow 2n$) and return to Step 1. If $n \geq |l|$, terminate and return the current l as the minimal failure-inducing subset.

A defining characteristic of DDMIN is its uniform partitioning strategy. By dividing the input into partitions of equal length (i.e., the same number of elements), the algorithm treats every element as interchangeable, overlooking significant variations in size or complexity. To illustrate this process, Figure 3 shows the step-by-step reduction trace of DDMIN on our motivating example. Each row corresponds to a candidate test case constructed during an iteration, with included elements highlighted in cyan. The bottom row denotes whether the test case passes the predicate ψ (i.e., still exhibits the bug): T for true (interesting) and F for false (not interesting).

2.4 Probabilistic Delta Debugging (ProbDD)

ProbDD [24] augments ddmin by replacing its deterministic partitioning with a probability-guided search. Instead of treating all elements equally, ProbDD maintains a probability score for each element, reflecting its estimated likelihood of being essential to satisfy the predicate ψ. These probabilities are iteratively updated based on test outcomes using Bayesian updating principles.

At each iteration, elements are sorted in ascending order by their current probabilities. ProbDD selects a prefix of this ordering that maximizes the expected number of deletable elements, calculated as: $\text{ExpectedDeletable}(pre) = |pre| \cdot \prod_{e \in pre}(1 - \text{prob}[e])$, where pre is the candidate subset and $\text{prob}[e]$ denotes the current probability of element e being essential.

If removing pre satisfies ψ, the elements are deleted, and their probabilities are set to 0. Otherwise, their probabilities are updated via: $\text{prob}[e] \leftarrow \frac{\text{prob}[e]}{1-\prod_{e' \in pre}(1-\text{prob}[e'])}$.

This learning process lets ProbDD target low-probability elements early and refine its model over time, often reducing inputs more efficiently than ddmin. However, like ddmin, it assumes uniform element cost and ignores structural factors such as size, depth, or complexity.

2.5 Weighted Delta Debugging (WDD)

WDD [39] addresses element size disparity by introducing token-based weighted partitioning. It assigns each element a weight $w(e)$ equal to its token count and modifies traditional delta debugging to create partitions balanced by total weight rather than element count. For the program in Figure 1, the eight statements have token counts of 5, 16, 16, 8, 8, 7, 25, and 6, representing a relatively narrow range that limits the effectiveness of token-based differentiation.

WDD realizes this concept through the following two algorithm variants:

- **Weighted ddmin (W_{ddmin}):** Extends ddmin with weight-balanced partitioning while maintaining the same three-step structure. The key modification is the `weightedPartition` function that splits partitions to minimize weight variance: $\sum_{e \in p_i} w(e) \approx \sum_{e \in p_j} w(e)$ for all partitions p_i, p_j. An additional `ensureOneMinimal` pass guarantees 1-minimality by individually testing remaining elements.
- **Weighted ProbDD (W_{ProbDD}):** Enhances ProbDD by incorporating weights into its probabilistic selection model. It sorts elements by expected weight reduction ($w(e) \times (1 - \text{prob}[e])$) and redefines the gain function as: $\text{Gain}(m) = \sum_{i=1}^{m} w_i \cdot \prod_{j=1}^{m}(1 - p_j)$, where m is the number of elements to remove, w_i the weight of the i-th element, and p_j the retention probability. This maximizes expected weight reduction rather than element count.

Both variants retain the original complexity but improve balance through token-awareness, a clear advance over uniform methods. However, token counting is a syntactic approximation: a simple declaration and a complex function may have similar counts yet vastly different roles in program behavior. This limitation is evident in Figure 1, where W_{ddmin} and ddmin (along with W_{ProbDD} and ProbDD) yield identical reduction traces (Figure 3) due to uniform token distribution, underscoring the need for more nuanced structural analysis beyond surface-level metrics. These observations motivate a shift from syntactic proxies to explicit structural modeling. Rather than approximating complexity via token volume, we directly quantify structural properties of AST subtrees and use them to guide reduction. We present this structure-aware framework below.

3 Structure-Aware Delta Debugging

We present Structure-Aware Delta Debugging (SADD), a novel approach that enhances delta debugging algorithms by incorporating structural characteristics of AST elements into the reduction process. SADD assigns each element a structural weight $w_{\text{struct}}(e)$ based on its geometric and information-theoretic complexity within the syntax tree, guiding partitioning and selection in both

Table 1. Structural metrics for all elements in our motivating example (Figure 1). Geometric Volume $V(e)$, Decision Uniformity $U(e)$, and Effective Branching Complexity $B(e)$ are computed from AST structures (Figure 2). $w_{\text{struct}}(e)$ denotes the resulting structural weight.

Line #	Depth	Leaves	V	U	B	$\varphi(B)$	Tokens	w_{struct}
1	1	2	2	1.0000	1.0000	0.0588	5	5.1176
2	3	4	12	0.9643	1.5523	0.0884	16	25.6324
3	3	4	12	0.9643	1.5523	0.0884	16	25.6324
4	3	5	15	0.9156	1.7415	0.0982	8	31.2073
5	3	5	15	0.9156	1.7415	0.0982	8	31.2073
6	3	4	12	0.9643	1.5523	0.0884	7	25.6324
7	4	11	44	0.8892	1.6705	0.0945	25	88.2828
8	1	3	3	1.0000	2.0000	0.1111	6	7.3333

DDMIN- and PROBDD-style reduction. While SADD can substantially accelerate existing techniques, its benefits are not uniform across all inputs. In this section, we introduce the structural weight function (Section 3.1), define its three constituent metrics—*geometric volume*, *decision uniformity*, and *branching complexity*—(Section 3.2), show how these weights drive two practical algorithms, SA_{DDMIN} and $\text{SA}_{\text{PROBDD}}$ (Section 3.3), and revisit our motivating example to illustrate how structural weighting produces different reduction traces from token-based strategies (Section 3.4).

3.1 Unified Structural Weight Function

We formalize the core of the SADD framework: a unified structural weight function $w_{\text{struct}}(e)$. Unlike one-dimensional token-based weighting, it quantifies the intrinsic complexity of each AST element e (subtree T_u rooted at node u) across three dimensions: geometric extent, distributional balance, and decision complexity. The full formulation is:

$$w_{\text{struct}}(e) = V(e) \cdot \Big(1 + \rho_U\, U(e) + \rho_B\, \varphi(B(e); C)\Big) + 1 \tag{1}$$

where $V(e)$, $U(e)$, and $B(e)$ are the metrics for geometric volume, decision uniformity, and effective branching complexity, respectively, which are detailed in Section 3.2. The parameters $\rho_U, \rho_B \in [0, 1]$ are hyperparameters controlling the influence of the structural corrections, while the saturation function $\varphi(x; C) = \frac{x}{x+C}$ prevents the branching complexity term from having an excessive impact.

This multiplicative design keeps geometric volume as the dominant component while letting entropy and branching measures provide proportional adjustments. It addresses limitations of token-based approaches: elements with different shapes but similar token counts may receive inappropriate weighting, balanced and skewed branching structures are not distinguished, and decision points with varying arity are treated uniformly. The +1 offset ensures non-zero weights so that even minimal leaf nodes receive weight 1, preserving algorithmic compatibility. We note that $w_{\text{struct}}(\cdot)$ is a purely structural heuristic computed from the program's tree representation; it does not directly measure semantic importance.

3.2 Structural Complexity Metrics

We now define each component of our structural weight function and demonstrate how each is computed on examples from our motivating program (Figure 1) and its corresponding AST (Figure 2). The computed metrics for all eight top-level nodes are summarized in Table 1. Each metric addresses specific limitations of prior work while offering complementary structural insights.

3.2.1 Geometric Volume $V(e)$. We begin with a geometric metric that quantifies the *spatial extent* of the AST subtree T_u rooted at node u:

$$V(e) = \text{depth}(u) \cdot \text{leaves}(u) \tag{2}$$

Here, depth(u) denotes the maximum path length from the subtree's root u to any of its leaves, and leaves(u) is the total count of terminal nodes in the subtree. This product provides a simple, geometry-aware measure that captures structural scale beyond mere token enumeration.

Examples. To illustrate this calculation, consider two representative nodes. The simple 'typedef' (L1), with a depth of 1 and 2 leaves, has a small volume of $V(\text{L1}) = 2$. In contrast, the complex function definition (L7), with a depth of 4 and 11 leaves, yields a much larger volume of $V(\text{L7}) = 44$. These cases show that different elements can have vastly different structural scales. The complete volume calculations for all eight top-level nodes are given in Table 1.

Limitation: While $V(e)$ captures overall structural extent, it cannot distinguish between fundamentally different geometric shapes. For example, a shallow, wide tree and a deep, narrow tree may yield identical volumes despite having very different internal structures. As evident in Table 1, this geometric foundation provides the dominant component of our structural weights, but the distributional characteristics require additional analysis, which motivates our next metric.

3.2.2 *Decision Uniformity* $U(e)$.

To address the shape-blindness of geometric volume, this second metric quantifies the *distributional balance* of a subtree.

Definition 3.1. A decision node is any non-terminal AST node with two or more children, representing points where structural complexity branches.

In Figure 2, all decision nodes are highlighted in orange.

We quantify the structural balance of a decision node using normalized Shannon entropy [19]. Let $\mathcal{D}(u)$ denote the decision nodes in subtree T_u. We define the uniformity of element (subtree) e, denoted $U(e)$, as the leaf-weighted average of the normalized entropies of its decision nodes:

$$U(e) = \frac{\sum_{d \in \mathcal{D}(u)} \text{leaves}(d) \cdot (H(d)/\ln k_d)}{\sum_{d \in \mathcal{D}(u)} \text{leaves}(d)} \tag{3}$$

where k_d is the number of children of decision node d, $H(d) = -\sum_{i=1}^{k_d} p_i \ln p_i$ is the node's entropy, and $p_i = \text{leaves}(c_i)/\sum_j \text{leaves}(c_j)$ is the relative leaf mass of child c_i. The resulting $U(e)$ value is in the range $[0, 1]$, where values near 1 indicate highly balanced branching (e.g., a balanced binary tree) and values near 0 indicate skewed structures (e.g., a linked list).

Examples. The simple 'typedef' (L1) consists of a single decision node with two equally-sized leaf children, resulting in a perfectly balanced structure with maximum uniformity, $U(\text{L1}) = 1.0$. In contrast, the complex function definition (L7) contains multiple decision nodes with varied child distributions (Figure 2), resulting in a lower overall uniformity of $U(\text{L7}) = 0.8892$.

Limitation: While this metric captures distributional shape, the normalization erases scale information. A balanced 2-way branch and a balanced 20-way branch both yield a normalized entropy of 1, despite having vastly different branching complexities. This motivates our final metric.

3.2.3 *Effective Branching Complexity* $B(e)$.

While $U(e)$ captures distributional balance, it normalizes away the absolute scale of decision complexity. To restore *scale sensitivity* while retaining distributional awareness, we adopt a perplexity-based analysis from information theory [14, 19].

Background: Perplexity. Perplexity is a standard transformation of Shannon entropy, defined as $e^{H(d)}$, which quantifies the "effective number of choices" at a decision point. It is widely used in language modeling and decision tree analysis to approximate the average branching factor.

We define the effective branching complexity of an element e as follows:

$$B(e) = \frac{\sum_{d \in \mathcal{D}(u)} \text{leaves}(d) \cdot (e^{H(d)} - 1)}{\sum_{d \in \mathcal{D}(u)} \text{leaves}(d)} \tag{4}$$

The subtraction of 1 removes the baseline case of a single-child branch (which yields entropy zero and perplexity 1), isolating the excess complexity that affects reduction difficulty.

Examples. The simple 'typedef' (L1), with its single binary decision point, has an effective branching load of $B(\text{L1}) = 1.0$. In contrast, the function definition (L7) involves multiple, more complex decision points, yielding a higher branching load of $B(\text{L7}) = 1.6705$, quantifying the increased navigational complexity that a reduction algorithm must face.

3.2.4 Summary of Metrics. Our three metrics are complementary, each addressing a distinct aspect of structural complexity:

- **Geometric Volume (V)** measures the overall spatial scale of an element but can conflate different structural shapes.
- **Decision Uniformity (U)** captures the distributional balance of an element's structure but omits information about its scale.
- **Effective Branching Complexity (B)** restores this scale sensitivity by interpreting entropy as the effective number of choices, quantifying the complexity of an element's decision points.

Together, these metrics enable our unified weight function to distinguish program elements more effectively than token counts alone.

3.3 Algorithmic Instantiations

We present SA_{DDMIN} (structure-aware DDMIN [34]) and $\text{SA}_{\text{ProbDD}}$ (structure-aware ProbDD [24]), which incorporate our structural weight function $w_{\text{struct}}(\cdot)$ from Equation (1) with minimal algorithmic changes. Although both variants use $w_{\text{struct}}(\cdot)$, its role differs: in SA_{DDMIN}, weights determine partition boundaries (weight-balanced partitioning); in $\text{SA}_{\text{ProbDD}}$, weights redefine the gain objective for candidate selection. In both cases, we preserve control structure and termination; weights affect only how inputs are grouped or candidates ranked.

3.3.1 SA_{DDMIN}: Structure-Aware DDMIN. DDMIN uses a classic binary search strategy that repeatedly divides an element list l into n partitions of equal size, tests subsets and complements, and then increases granularity. SA_{DDMIN} retains this search structure but replaces uniform element-count partitioning with weight-balanced partitioning guided by structural complexity. The key insight is that element-count-balanced partitioning can distribute structural complexity unevenly, since elements within the same partition may contribute very differently to reduction difficulty as approximated by $w_{\text{struct}}(\cdot)$. SA_{DDMIN} therefore constructs *structural-weight-balanced* partitions so that each carries comparable total complexity. This preserves DDMIN's divide-and-conquer workflow while ensuring that granularity refinement operates over partitions aligned with the intended notion of complexity, leaving the subset/complement exploration unchanged.

Weight-Balanced Partitioning Strategy. To create partitions balanced by structural weights, we use a greedy algorithm. Given a list of elements and a target partition count n, we first compute the total structural weight $W = \sum_{i=1}^{m} w_{\text{struct}}(e_i)$ and the ideal target weight per partition $W^* = W/n$. The algorithm then iterates through the elements, adding them to the current partition until including the next element would make its cumulative weight exceed W^*. At that point, the current partition is finalized, and a new one is started. This process produces partitions with balanced structural complexity rather than simply balanced element counts.

Algorithm Integration. SA_{DDMIN} preserves the overall control flow of DDMIN [34] (Section 2.3), retaining its three-phase reduction strategy:

(1) **Reduce to Subset:** Evaluate each weight-balanced partition independently.

(2) **Reduce to Complement:** Evaluate the complement of each partition.
(3) **Granularity Adjustment:** Double the number of partitions and repeat.

The only change from DDMIN lies in the partitioning mechanism: instead of dividing by element count, SA_{DDMIN} creates partitions balanced by structural weight. All other aspects—test ordering, granularity updates, and termination conditions—remain unchanged. Following W_{DDMIN} [39], SA_{DDMIN} includes a final `ensureOneMinimal` pass that individually tests each remaining element, guaranteeing 1-minimality.

3.3.2 $\text{SA}_{\text{ProbDD}}$: Structure-Aware Probabilistic Selection. ProbDD [24] (Section 2.4) maintains probability estimates for each element and selects subsets that maximize expected deletion gain. $\text{SA}_{\text{ProbDD}}$ augments this framework by incorporating structural weights into the gain function, prioritizing structurally complex elements for removal. Unlike SA_{DDMIN}, which modifies grouping, $\text{SA}_{\text{ProbDD}}$ modifies the objective used to rank candidate subsets. In ProbDD and W_{ProbDD}, gain is defined in terms of element or token count, so subsets with very different structural complexity may receive comparable scores. $\text{SA}_{\text{ProbDD}}$ instead defines gain using $w_{\text{struct}}(\cdot)$, ensuring that ranking reflects structural complexity rather than surface-level size; the objective is thus grounded in the structural components of $w_{\text{struct}}(\cdot)$ (e.g., V, U, and B) rather than a token-count proxy. Importantly, ProbDD's Bayesian update rule remains unchanged: structural weighting affects only candidate ranking, while probability estimates are refined as in the original algorithm.

Gain Function. In the original ProbDD, the selection score for a candidate subset *pre* is:

$$\text{Gain}(pre) = |pre| \cdot \prod_{e \in pre} (1 - p_e) \tag{5}$$

where p_e is the estimated probability that element e must be retained. To incorporate structural complexity, $\text{SA}_{\text{ProbDD}}$ replaces the cardinality term with cumulative structural weight:

$$\text{Gain}_{\text{struct}}(pre) = \left(\sum_{e \in pre} w_{\text{struct}}(e) \right) \cdot \prod_{e \in pre} (1 - p_e) \tag{6}$$

This revised objective favors subsets whose removal eliminates greater structural complexity while preserving ProbDD's probabilistic modeling of element indispensability.

Selection Strategy. $\text{SA}_{\text{ProbDD}}$ ranks elements by their expected individual gain $w_{\text{struct}}(e)(1-p_e)$ and selects the prefix that maximizes $\text{Gain}_{\text{struct}}(pre)$. The probability update rule p_e remains unchanged, so structural weighting affects only candidate selection without modifying the learning mechanism.

3.3.3 Practical Considerations. SADD augments its base algorithms with guided partitioning (SA_{DDMIN}) or guided prioritization ($\text{SA}_{\text{ProbDD}}$) while leaving the search space and core delta debugging process unchanged. Both instantiations preserve the key properties of their base algorithms:

- **Output validity.** Each candidate reduction is validated by the same oracle as in DDMIN and ProbDD. Structural weights affect only how candidates are constructed or ranked, not how outcomes are evaluated; thus the final output remains failure-inducing under the original criterion.
- **Minimality.** SA_{DDMIN} retains DDMIN's three-phase reduction logic and, as in W_{DDMIN} [39], includes the same `ensureOneMinimal` pass that individually tests remaining elements, guaranteeing 1-minimality. $\text{SA}_{\text{ProbDD}}$ inherits ProbDD's conditional 1-minimality under the assumption of element independence [24], as it preserves the original Bayesian update rule and termination conditions, thereby matching the base reducer's minimality criterion.

Inputs	l_1	l_2	l_3	l_4	l_5	l_6	l_7	l_8	l_9	l_{10}	l_{11}	l_{12}	l_{13}	l_{14}	l_{15}	l_{16}	l_{17}	l_{18}	l_{19}	l_{20}	l_{21}	l_{22}	l_{23}	l_{24}	l_{25}	l_{26}
Initial Elements	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1	e_1
	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2	e_2
	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3	e_3
	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4	e_4
	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5	e_5
	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6	e_6
	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7	e_7
	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8	e_8
ψ	F	F	F	F	F	F	F	**T**	F	F	F	F	F	F	F	F	F	F	F	F	F	**T**	F	F	F	F

Fig. 4. Step-by-step trace of SA_{DDMIN} on the motivating example in Figure 1. Each step includes a test case with statements highlighted in cyan. Test cases are flagged '**T**' if they satisfy ψ and 'F' otherwise.

- **Time complexity.** SA_{DDMIN} preserves the $O(n^2)$ worst-case query complexity of DDMIN [34], and SA_{ProbDD} preserves the $O(n)$ worst-case query complexity of ProbDD [24] (where n is the number of elements), as both modify only the weighting signal without altering the control logic or search space. Computing $w_{struct}(\cdot)$ adds a one-time $O(|AST|)$ preprocessing cost.

3.4 Worked Example: Revisiting GCC-71626

We revisit our motivating example in Figure 1 to illustrate how SA_{DDMIN} leverages structural complexity to improve reduction efficiency over DDMIN and W_{DDMIN}.

Using the weights from Table 1, SA_{DDMIN} performs an asymmetric first split: the low-weight partition $\{L1, L2, L3, L4, L5\}$ has weight **118.8**, while the high-weight partition $\{L6, L7, L8\}$ has **121.2**. This places the most complex element $L7$ ($w_{struct} = 88.28$) in a smaller, high-weight partition while keeping the two partition weights nearly balanced. In contrast, both DDMIN and W_{DDMIN} use a uniform 4/4 split: $\{L1, L2, L3, L4\}$ and $\{L5, L6, L7, L8\}$. DDMIN does so by element count, while W_{DDMIN}—despite token-based weighting—produces almost identical token totals (**45** vs. **46**), offering no more guidance than DDMIN in this case.

In our approach, structural guidance accelerates the reduction process. As shown in Figure 4, SA_{DDMIN} completes the reduction in 26 queries, compared to 30 for both DDMIN and W_{DDMIN} to reach the same 1-minimal output. The effect is even stronger at scale: for the full GCC-71626 case under the HDD framework (Table 2), DDMIN requires over 73,000 queries, W_{DDMIN} about 17,000, and SA_{DDMIN} just 9,300—an almost 8× reduction over the classic DDMIN approach.

Additionally, Table 1 illustrates how our approach captures structural complexity beyond surface-level metrics. Although L4 has half the token count of L3 (8 vs. 16), it receives a higher structural weight ($w_{struct}(L4) = 31.21$ vs. $w_{struct}(L3) = 25.63$). This difference reflects their distinct structural roles: L4 defines a function that introduces a new scope and execution context, whereas L3 declares a type alias that performs static type binding.

4 Evaluation

We evaluate SADD against three established delta debugging approaches—DDMIN, ProbDD, and WDD—to assess its effectiveness and efficiency. Our goal is to determine when structure-aware weighting yields measurable gains over uniform and token-based partitioning, and to identify cases where its benefits are limited or absent. We address the following four research questions:

- **RQ1:** How does SA_{DDMIN} perform compared to DDMIN?
- **RQ2:** How does SA_{ProbDD} perform compared to ProbDD?
- **RQ3:** How does SADD perform compared to WDD?
- **RQ4:** What is the contribution of each structural component (V, U, B) to SADD performance?

4.1 Methodology

Having introduced a structural weighting model that preserves the underlying search control, we now evaluate whether this modeling refinement translates into measurable efficiency gains in practice. We instantiate SADD in two widely used tree-based reduction frameworks, HDD [15] and Perses [21], and compare the following algorithmic variants within each framework:

- **Baseline algorithms:** ddmin and ProbDD
- **Token-based weighting:** WDD variants (W_{ddmin} and W_{ProbDD})
- **Structure-aware weighting:** SADD variants (SA_{ddmin} and SA_{ProbDD})

We use the open-source implementations of HDD and Perses [17], integrating our structure-aware algorithms with minimal changes to their workflows. All algorithms run in fixpoint mode to achieve 1-tree-minimality, following standard practice in hierarchical reduction [15, 21, 39].

For SADD, we set $\rho_U = \rho_B = 1$ and $C = 16$ in Equation (1) and use this single configuration uniformly across all benchmarks and both language domains. This avoids per-benchmark tuning, ensuring a fair comparison with WDD, and demonstrates that SADD operates effectively with a fixed, out-of-the-box setting. Assigning $\rho_U = \rho_B = 1$ gives equal emphasis to all three structural components, while the saturation function $\varphi(\cdot\,; C{=}16)$ limits the influence of extremely high branching factors. We analyze the contribution of individual structural components via ablation in Section 4.6 (with C fixed).

4.2 Experimental Setup

4.2.1 Subjects. We use the same benchmark suite as WDD [39] to enable direct result comparison. It comprises 62 real-world programs in two programming languages:

- **C Programs:** 32 programs from prior studies [21, 23, 38] that trigger real bugs in LLVM and GCC. These are large and complex, averaging 77,723 tokens and representing realistic compiler testing scenarios.
- **XML Files:** 30 XML documents that trigger bugs in BaseX [1], a widely used XML database and XQuery processor, averaging 20,197 tokens.

This mix of C (general-purpose) and XML (DSL markup) allows us to assess our structural analysis across contrasting syntactic paradigms.

4.2.2 Metrics. Following standard practice in delta debugging evaluation [21, 24, 39], we measure:

- **Reduction Time (T(s)):** Total time taken for the reduction process, in seconds
- **Output Size (S(#)):** Number of tokens in the final reduced program
- **Test Queries (Q(#)):** Number of property tests actually executed

For significance testing, we use the Wilcoxon signed-rank test [27] with a $p < 0.05$ threshold, following standard practice in software engineering studies. We report geometric rather than arithmetic means, as subjects vary widely on these metrics.

4.2.3 Computing Platform. All experiments ran on an Ubuntu 22.04 server with an Intel Xeon CPU @ 3.20GHz and 256 GB RAM, using single-process, single-threaded execution for fair comparison.

4.3 RQ1: SA_{ddmin} vs. ddmin

To evaluate the impact of structure-aware weighting, we compare SA_{ddmin} with the standard ddmin algorithm in both HDD and Perses. Table 2 presents results across 62 benchmarks.

4.3.1 Effectiveness. SA_{ddmin}'s effectiveness varies by framework and input. In HDD, reduced programs average 1.56% smaller on C and 28.86% smaller on XML ($p = 0.002$). In Perses, changes are negligible: 2.15% larger on C and 0.19% smaller on XML ($p = 0.677$). This disparity reflects

Table 2. Results of SA_{DDMIN} compared to DDMIN and W_{DDMIN} across the HDD and PERSES frameworks on all 62 C and XML benchmarks. The better result in each comparison is shown in bold.

Benchmark	HDD									PERSES								
	DDMIN			W_{DDMIN}			SA_{DDMIN}			DDMIN			W_{DDMIN}			SA_{DDMIN}		
	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)
clang-18596	12,968	452	86158	9,086	**414**	48812	**8,624**	502	**43532**	5,049	**260**	5316	**4,963**	**260**	4893	5,039	**260**	**4789**
clang-19595	9,978	285	65855	7,654	**260**	35358	**7,439**	267	**31698**	4,137	**156**	3819	**4,049**	**156**	3560	4,096	**156**	**3460**
clang-20680	18,178	377	101869	**12,285**	366	**53971**	12,518	**315**	57550	14,336	**525**	9173	12,720	**525**	8022	**12,546**	537	**7978**
clang-21467	16,785	455	118873	10,735	**415**	57086	**10,118**	**415**	**52375**	6,431	**177**	6200	6,413	178	5779	**6,187**	**177**	**5775**
clang-21582	16,881	1197	124000	10,522	**999**	78238	**9,609**	1164	**77558**	7,100	**559**	10077	8,076	**559**	10826	**7,028**	**559**	**9554**
clang-22337	15,627	**299**	100901	7,647	323	50012	**6,508**	311	**43818**	**2,493**	**236**	6727	2,627	**236**	6196	2,496	**236**	**6165**
clang-22382	6,259	**182**	91087	3,317	194	48986	**2,537**	194	**39339**	814	**144**	5190	719	**144**	4896	**681**	**144**	**4663**
clang-22704	23,793	**95**	80275	16,630	113	54709	**11,482**	**95**	**45141**	6,023	**78**	6205	5,510	**78**	5349	**5,209**	**78**	**5207**
clang-23309	25,262	1086	162065	11,006	1067	88648	**8,185**	**1008**	**74526**	2,408	**475**	8323	**2,262**	**475**	7816	2,344	**475**	**7632**
clang-23353	16,774	**144**	147651	9,371	153	86215	**7,763**	153	**69959**	**1,411**	**98**	7320	1,669	**98**	**6342**	1,482	113	6736
clang-25900	15,836	478	106939	4,492	**346**	50040	**3,853**	501	**39656**	1,223	**252**	5286	**1,102**	**252**	4790	2,143	**252**	**4766**
clang-26350	56,300	391	117835	35,707	**378**	79156	**23,110**	612	**65490**	**7,132**	193	**7283**	8,450	**189**	8363	7,508	193	7935
clang-26760	32,379	321	72139	7,871	**229**	43287	**5,631**	261	**32163**	4,331	**91**	4242	3,613	**91**	**3836**	**3,179**	**91**	3886
clang-27137	292,484	582	259586	91,044	545	116131	**69,076**	**514**	**100969**	15,773	**268**	9650	14,501	**268**	**8704**	**13,296**	325	9084
clang-27747	3,076	299	41828	**1,961**	**244**	24715	2,072	277	**23578**	1,561	**117**	4075	**1,169**	**117**	3298	1,174	**117**	**3146**
clang-31259	17,456	556	114893	9,516	594	69595	**7,875**	**541**	**54013**	2,728	**384**	4981	**2,645**	**384**	**4600**	2,711	**384**	4609
gcc-58731	8,965	431	49486	6,069	**375**	29939	**5,942**	396	**26602**	**3,273**	**213**	5342	3,379	**213**	4804	3,406	**213**	**4651**
gcc-59903	16,400	**545**	116611	8,671	734	70439	**7,637**	683	**54579**	5,093	497	7721	4,549	**382**	7270	**4,266**	**382**	**7183**
gcc-60116	18,809	1281	145785	10,247	1137	88968	**8,985**	**1015**	**71388**	3,080	**443**	9690	3,090	**443**	9039	**3,069**	**443**	**8836**
gcc-60452	20,945	491	130676	8,763	**442**	64386	**7,532**	523	**55250**	2,368	350	7775	**2,259**	350	7147	2,581	**342**	**7082**
gcc-61047	8,360	**495**	81936	4,573	505	48283	**3,838**	505	**43710**	1,274	**266**	**3248**	**997**	**266**	3295	1,094	**266**	3275
gcc-61383	13,997	579	107994	6,982	**421**	58988	**5,847**	441	**50429**	**3,095**	**271**	6354	3,149	274	5604	3,440	274	**5461**
gcc-61917	14,933	**293**	93218	7,966	311	48138	**6,471**	311	**40892**	2,622	**142**	6352	1,779	**142**	5470	**1,765**	**142**	**5327**
gcc-64990	35,836	**325**	87929	23,447	378	61845	**19,599**	386	**50239**	**4,719**	**239**	6556	4,911	**239**	6183	4,745	**239**	**5859**
gcc-65383	12,405	246	100767	6,211	**217**	52651	**5,588**	240	**40804**	1,411	**153**	4411	**1,337**	**153**	3992	1,345	**153**	**3961**
gcc-66186	14,244	605	88978	7,656	**508**	45864	**7,497**	559	**44658**	**2,918**	**327**	5319	2,932	**327**	5083	3,059	**327**	**5044**
gcc-66375	58,573	1053	169003	21,762	**740**	83372	**21,498**	865	**75957**	**3,775**	**440**	6717	3,891	**440**	6386	4,098	**440**	**6241**
gcc-66412	19,806	491	126092	8,005	**442**	64366	**7,115**	523	**55164**	2,423	350	7775	**2,311**	350	7147	2,874	**342**	**7104**
gcc-66691	16,094	1076	121346	10,746	**1022**	79773	**9,060**	1056	**68695**	3,774	746	9465	**3,655**	**602**	**8007**	4,008	746	9022
gcc-70127	85,080	617	145452	34,324	576	67060	**28,410**	**538**	**58857**	4,774	**301**	5545	**4,546**	**301**	5147	4,633	**301**	**5086**
gcc-70586	82,061	**792**	127760	49,882	793	94138	**39,701**	833	**85526**	**8,374**	**197**	7507	8,500	367	7539	9,359	367	**7443**
gcc-71626	2,370	**53**	73801	480	**53**	17401	**332**	**53**	**9305**	84	**51**	**1007**	64	**51**	1016	**63**	**51**	1010
Mean	18,796	422	104682	9,487	**396**	57171	**8,060**	416	**48529**	3,074	**236**	5958	**2,914**	237	5530	2,976	241	**5459**
xml-1	768	33	294	**509**	**24**	**206**	542	**24**	**206**	170	**16**	**53**	**169**	**16**	54	170	**16**	54
xml-2	1,826	60	740	**238**	**15**	**139**	247	**15**	**139**	**46**	**15**	**18**	47	**15**	**18**	48	**15**	**18**
xml-3	794	36	324	**292**	**15**	**139**	313	**15**	**139**	**61**	**15**	**20**	**61**	**15**	**20**	62	**15**	21
xml-4	**2,499**	**78**	949	2,506	**78**	948	2,744	**78**	**946**	794	**13**	202	**792**	**13**	203	794	**13**	**200**
xml-5	1,047	33	435	**256**	**15**	**129**	276	**15**	**129**	61	**15**	**20**	**60**	**15**	**20**	**60**	**15**	21
xml-6	3,890	**120**	**1497**	**3,793**	**120**	**1497**	4,028	**120**	**1497**	**405**	**30**	**107**	427	**30**	109	451	**30**	108
xml-7	**1,897**	**69**	**875**	1,905	**69**	**875**	2,009	**69**	878	443	**33**	159	**426**	**33**	**155**	429	**33**	156
xml-8	3,405	138	1718	**418**	**24**	**197**	443	**24**	**197**	259	**16**	**84**	256	**16**	85	**245**	**16**	85
xml-9	1,720	**63**	839	**1,585**	**63**	834	1,846	**63**	**833**	**891**	**37**	277	901	**37**	**273**	920	**37**	**273**
xml-10	3,056	**102**	1245	**2,970**	**102**	**1237**	3,251	**102**	**1237**	953	**37**	248	**941**	**37**	248	953	**37**	**246**
xml-11	1,774	90	727	**468**	**24**	**198**	524	**24**	**198**	279	**16**	**85**	281	**16**	86	**277**	**16**	86
xml-12	1,493	96	762	**1,302**	**87**	**667**	1,373	**87**	**667**	**280**	**16**	**124**	289	**16**	**124**	298	**16**	125
xml-13	1,300	**78**	630	**1,276**	**78**	**620**	1,338	**78**	623	324	**25**	125	**321**	**25**	**123**	**321**	**25**	**123**
xml-14	3,796	**153**	1837	**3,780**	**153**	**1828**	3,993	**153**	1831	1,032	**43**	389	1,008	**43**	**388**	**1,005**	**43**	**388**
xml-15	2,093	108	878	**927**	**51**	**400**	1,033	**51**	**400**	335	**16**	105	316	**16**	**104**	**315**	**16**	**104**
xml-16	1,628	60	730	**479**	**24**	**224**	513	**24**	**224**	304	**16**	**97**	296	**16**	98	**290**	**16**	98
xml-17	2,263	87	1016	**277**	**15**	**146**	282	**15**	**146**	70	**15**	26	**69**	**15**	**25**	**69**	**15**	26
xml-18	732	**39**	313	**696**	**39**	**307**	770	**39**	**307**	288	**16**	91	283	**16**	91	**282**	**16**	**90**
xml-19	**1,481**	**54**	**739**	1,553	**54**	745	1,662	**54**	748	439	**36**	165	**438**	**36**	**162**	443	**36**	**162**
xml-20	2,415	**99**	860	**2,379**	**99**	**845**	2,481	**99**	848	1,427	**64**	**367**	1,416	**64**	369	**1,393**	**64**	371
xml-21	3,348	93	1249	**3,207**	**90**	**1168**	3,343	**90**	**1168**	987	**46**	257	**931**	**46**	**242**	977	**46**	**242**
xml-22	1,622	**61**	760	**1,603**	**61**	**739**	1,662	**61**	747	261	**24**	100	**251**	**24**	**98**	259	**24**	99
xml-23	7,931	189	2673	**6,517**	**165**	**2184**	7,014	**165**	2192	1,596	54	383	1,565	**51**	380	**1,556**	**51**	**379**
xml-24	**2,415**	**96**	900	2,438	**96**	**895**	2,661	**96**	903	1,401	**70**	382	1,401	**70**	**381**	**1,395**	**70**	382
xml-25	**4,312**	135	**1439**	5,823	**132**	2006	6,276	**132**	2013	1,432	**55**	360	**1,413**	**55**	**357**	1,415	**55**	358
xml-26	**6,570**	**126**	2027	6,635	**126**	2028	7,167	**126**	**2026**	**1,786**	**64**	**431**	1,810	**64**	436	1,826	**64**	440
xml-27	9,367	**195**	2912	**9,161**	**195**	**2873**	9,869	**195**	**2873**	**2,884**	**102**	712	2,901	**102**	713	2,909	**102**	**710**
xml-28	8,021	**159**	2708	**7,863**	**159**	2673	8,403	**159**	**2655**	2,521	**97**	**630**	2,529	**97**	633	**2,502**	**97**	634
xml-29	6,532	147	2273	**5,593**	**138**	**1974**	6,030	**138**	1978	1,308	**48**	334	1,297	**48**	**329**	**1,294**	**48**	**329**
xml-30	6,257	**147**	2143	**6,002**	**147**	2120	6,768	**147**	**2118**	2,861	**78**	**700**	**2,859**	**78**	**700**	2,866	**78**	701
Mean	2,504	87	1012	**1,629**	**62**	**683**	1,750	**62**	**683**	504	**30**	**152**	**500**	**30**	**152**	503	**30**	**152**

architectural differences between the frameworks. HDD applies DDMIN hierarchically across all nodes, allowing our structural metrics to influence reductions globally. PERSES, by contrast, restricts DDMIN to *quantified nodes*—those with syntactically independent children produced by structural transformations (e.g., deletion and substitution) [21]—which removes the structural interactions our metrics (volume, branching, entropy) are designed to capture.

4.3.2 Efficiency. We report efficiency primarily in time, noting where helpful whether time reductions align with query reductions (e.g., 57.12% time vs. 53.64% queries—roughly proportional).

Strong Alignment in HDD-C: For C programs in HDD, SA_{DDMIN} cuts queries by 53.64% and time by 57.12% ($p = 7.95 \times 10^{-7}$). The similar query-to-time ratio suggests the removed queries are uniformly expensive, supporting the utility of structure-aware guidance. For example, on GCC-71626, SA_{DDMIN} reduces queries from 73,801 to 9,305—an 8× improvement.

Moderate Gains in HDD-XML: Query reduction is also notable for XML (32.46%, $p = 1.24 \times 10^{-4}$), with time savings remaining substantial at 30.12%, though smaller than in C, as XML's simpler, flatter structure leaves less structural complexity for SA_{DDMIN} to exploit.

Modest Gains in Perses-C: On C, SA_{DDMIN} reduces queries by 8.38% ($p = 6.59 \times 10^{-6}$) but yields only a 3.20% time improvement ($p = 0.286$). This modest effect reflects that Perses already handles expensive reductions efficiently through structural transformations before applying DDMIN, leaving mostly lightweight elements where structural guidance has limited impact.

Limited Impact in Perses-XML: On XML, SA_{DDMIN} yields nearly identical query counts (0.02% fewer, $p = 0.773$) and a marginal 0.30% time improvement ($p = 0.234$), suggesting that for XML's simpler, flatter structure under Perses —where structural transformations are already applied—structural weighting provides little additional benefit.

4.3.3 Analysis. These results reveal two key insights. First, framework design bounds the potential gains: when DDMIN is central to reduction, as in HDD, SA_{DDMIN} achieves substantial speedups (57.12% time savings), whereas when it plays only a peripheral role, as in Perses where structural transformations precede DDMIN, the benefits are smaller and sometimes slightly negative. Second, structural metrics are most effective when meaningful structural interactions remain—as in general-purpose languages like C, and to a lesser extent XML under HDD —but their impact diminishes when such structure is shallow or has been largely removed, as in XML under Perses.

RQ1: SA_{DDMIN} delivers its strongest gains when (1) DDMIN is central to reduction and (2) inputs exhibit substantial structural complexity. In the best case (C under HDD), it achieves 57.12% time savings with 1.56% smaller outputs. When structure is shallow or largely removed, as in XML under Perses, its impact diminishes, yielding only 0.30% time savings with comparable output quality. These bounds clarify where structural analysis adds value and where benefits taper off.

4.4 RQ2: SA_{ProbDD} vs. ProbDD

We evaluate SA_{ProbDD} against ProbDD [24] to assess whether structural weights enhance probabilistic selection. Because ProbDD is stochastic, results are averaged over five runs, unlike DDMIN (Section 4.3), which uses a single run following [39]. Table 3 shows the detailed results.

4.4.1 Effectiveness. In HDD, SA_{ProbDD} reduces output size by an average of 12.06% on C and 13.75% on XML compared to ProbDD ($p = 3.09 \times 10^{-7}$). These consistent reductions indicate that structural weighting effectively guides not only classic DDMIN-style reduction but also probabilistic selection.

In Perses, improvements are modest: 3.40% smaller on C and 0.20% larger on XML ($p = 0.241$). As noted in Section 4.3, Perses applies DDMIN only to quantified nodes (already structurally transformed), limiting the potential impact of structural weighting optimization.

4.4.2 Efficiency. Query-to-time ratios show how structural weights influence probabilistic selection. In HDD, both C and XML exhibit balanced improvements: C achieves 15.69% fewer queries with 14.64% less time ($p = 2.86 \times 10^{-4}$), while XML achieves 16.03% fewer queries with 20.10% less time ($p = 0.005$). These aligned ratios contrast with SA_{DDMIN}'s more divergent patterns (Section 4.3), suggesting that probabilistic selection naturally evens out query costs.

Table 3. Results of SA_{ProbDD} vs. ProbDD and W_{ProbDD} in HDD and Perses on 62 C and XML benchmarks. Metrics are averaged over five runs, and better results in each comparison are shown in bold.

Benchmark	HDD									Perses								
	ProbDD			W_{ProbDD}			SA_{ProbDD}			ProbDD			W_{ProbDD}			SA_{ProbDD}		
	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)	T(s)	S(#)	Q(#)
clang-18596	7,791	510	26287	6,560	482	21786	**6,503**	**462**	**19261**	4,992	264	3898	**4,708**	**260**	**3882**	4,795	**260**	3886
clang-19595	6,379	310	12625	**6,051**	**244**	**10557**	6,088	276	11695	4,301	**156**	**2743**	4,209	**156**	2784	**4,205**	**156**	2816
clang-20680	10,349	364	17912	8,634	**360**	**17319**	**8,197**	362	17327	12,363	**531**	7576	8,275	545	**6377**	**8,200**	539	6542
clang-21467	8,041	428	16502	7,077	**310**	**15312**	**6,924**	334	18010	6,284	**169**	**4934**	**6,083**	177	4978	6,085	177	4977
clang-21582	7,368	**1072**	49992	7,773	1079	43962	**6,630**	1106	**31920**	7,516	588	8819	6,554	**559**	**8361**	**6,420**	**559**	**8361**
clang-22337	2,777	320	14701	**2,559**	301	14532	2,672	**297**	**13369**	2,446	**249**	**5014**	**2,240**	250	5200	2,268	250	5198
clang-22382	1,094	203	11301	**974**	200	**10756**	1,005	**198**	11862	610	**144**	**3457**	**603**	**144**	3497	611	**144**	3497
clang-22704	9,384	114	10329	6,146	**90**	**8727**	**5,904**	103	9557	**4,362**	78	4209	8,107	**72**	**3884**	8,187	**72**	3886
clang-23309	5,915	1161	49278	4,449	**1088**	34376	**4,361**	1109	**31500**	1,887	**472**	**6376**	**1,766**	483	6520	1,793	483	6518
clang-23353	2,552	175	11843	**1,966**	218	11025	2,149	**134**	**10830**	1,494	149	5104	1,175	**98**	4518	**1,171**	**98**	**4516**
clang-25900	2,148	523	**17348**	2,102	471	22325	**2,094**	**467**	21110	971	**239**	3696	**907**	252	**3560**	962	252	3562
clang-26350	**16,132**	669	37071	16,572	457	31000	16,223	**418**	**28039**	6,977	**228**	5909	**6,411**	241	**5836**	6,472	241	5838
clang-26760	5,373	292	18287	3,867	276	18206	**3,682**	**252**	**12459**	3,006	113	3039	**2,776**	**90**	2890	2,782	**90**	**2884**
clang-27137	**26,913**	610	42333	27,642	643	**34495**	30,776	**592**	46304	11,437	**219**	6754	**10,916**	268	6734	11,039	268	**6731**
clang-27747	1,524	332	11489	**1,224**	302	**10574**	1,461	**298**	14701	1,500	**130**	2777	1,145	150	2610	**1,144**	150	**2609**
clang-31259	**4,686**	585	18296	5,995	571	20490	4,809	**548**	**17523**	2,534	402	3732	2,473	**384**	**3690**	**2,441**	**384**	3691
gcc-58731	6,297	418	11288	**4,103**	368	**7805**	5,368	**366**	12188	3,915	264	3288	3,128	**213**	**2531**	**3,110**	**213**	**2531**
gcc-59903	**4,675**	746	31387	6,255	**410**	**26201**	6,516	801	37725	4,577	417	5869	**3,633**	**300**	**5185**	3,758	**300**	5186
gcc-60116	6,933	1192	56837	**4,921**	**896**	**28645**	5,260	1092	31709	2,624	420	6330	2,498	**404**	**5995**	**2,471**	**404**	**5995**
gcc-60452	3,729	659	28523	**2,716**	**508**	**16625**	2,770	526	17842	2,295	**346**	6154	2,387	349	5815	**2,088**	350	**5808**
gcc-61047	**1,999**	565	15763	2,516	**512**	19548	2,074	538	**15649**	1,006	268	**2616**	958	**266**	2617	**942**	**266**	2619
gcc-61383	4,369	487	26642	3,981	510	25773	**3,772**	**440**	**21649**	**3,069**	282	5006	3,087	**274**	**4350**	3,100	**274**	4351
gcc-61917	3,643	310	14428	2,899	300	13227	**2,774**	**295**	**12593**	2,486	147	4507	**1,803**	**142**	**4230**	1,827	**142**	4232
gcc-64990	11,301	535	19160	10,599	467	**13983**	**10,558**	**407**	16555	4,596	**239**	4890	4,469	**239**	**4836**	**4,461**	**239**	4837
gcc-65383	2,387	278	14151	1,972	**275**	11592	**1,823**	315	**10806**	1,352	**153**	3125	1,295	**153**	3107	**1,244**	**153**	**3103**
gcc-66186	8,645	587	23857	7,552	503	24122	**5,166**	**461**	**13564**	3,091	**327**	4185	3,026	**327**	4148	**2,918**	**327**	**4147**
gcc-66375	11,787	837	26619	12,762	**813**	33444	**10,486**	827	**24130**	3,914	**440**	5038	3,501	**440**	4949	**3,474**	**440**	**4947**
gcc-66412	4,171	718	34753	2,956	525	18301	**2,817**	**513**	**18050**	2,299	**346**	5888	**2,145**	350	**5805**	2,201	350	5806
gcc-66691	4,598	947	27672	6,212	1039	30987	**3,346**	**566**	**20255**	3,679	680	7462	3,572	**603**	**6934**	**3,566**	**603**	6935
gcc-70127	18,447	666	37403	14,321	**640**	21292	**14,268**	675	**19439**	4,245	**301**	3959	3,880	**301**	3794	**3,829**	**301**	**3792**
gcc-70586	18,312	948	51140	**17,414**	718	**44401**	21,249	**627**	59019	7,105	201	**5640**	7,045	**197**	6272	**6,878**	**197**	6276
gcc-71626	213	**53**	3612	174	**53**	3143	**138**	**53**	**3121**	**61**	**51**	**747**	**61**	**51**	761	62	**51**	763
Mean	5,097	460	20999	4,545	411	18133	**4,351**	**404**	**17704**	2,797	242	4406	2,607	**234**	**4235**	**2,600**	**234**	4240
xml-1	664	35	200	**383**	**24**	**137**	412	**24**	138	308	**16**	**54**	296	**16**	**54**	**289**	**16**	**54**
xml-2	348	22	135	**164**	**15**	**84**	165	**15**	86	88	**15**	23	81	**15**	21	**74**	**15**	**20**
xml-3	298	19	113	**202**	**15**	**86**	216	**15**	88	**104**	**15**	**22**	107	**15**	23	109	**15**	23
xml-4	2,407	**78**	**681**	2,419	**78**	708	**2,406**	**78**	706	**1,277**	**13**	**206**	1,332	**13**	210	1,312	**13**	209
xml-5	386	20	139	**189**	**15**	**84**	193	**15**	85	88	**15**	**19**	**87**	**15**	**19**	88	**15**	**19**
xml-6	1,948	73	593	795	**30**	**279**	**770**	**30**	**279**	787	**30**	113	782	**30**	113	**757**	**30**	**112**
xml-7	**1,408**	**69**	**621**	1,566	**69**	629	1,546	**69**	628	688	**33**	**165**	**644**	**33**	167	669	**33**	167
xml-8	390	**24**	**137**	373	**24**	140	**361**	**24**	142	391	**16**	**84**	417	**16**	85	**365**	**16**	85
xml-9	2,555	79	722	2,209	90	827	**1,612**	**63**	**586**	1,952	**37**	**288**	1,925	**37**	292	**1,876**	**37**	292
xml-10	3,111	**102**	**879**	**2,974**	**102**	889	3,003	**102**	893	1,721	**37**	**256**	1,732	**37**	261	**1,709**	**37**	261
xml-11	683	37	228	**382**	**24**	**139**	407	**24**	141	453	**16**	85	448	**16**	85	**446**	**16**	**84**
xml-12	1,172	93	541	**1,083**	**87**	**511**	1,150	**87**	513	472	**16**	**131**	**454**	**16**	132	473	**16**	**131**
xml-13	**1,080**	**78**	**454**	1,082	**78**	460	1,117	**78**	466	**489**	**25**	**131**	501	**25**	134	505	**25**	134
xml-14	**3,074**	157	1341	3,239	**153**	1331	3,289	**153**	**1330**	**1,485**	**43**	**403**	1,488	**43**	405	1,505	**43**	405
xml-15	1,154	62	361	950	**51**	305	**905**	**51**	**304**	**516**	**16**	**110**	558	**16**	111	529	**16**	**110**
xml-16	951	38	310	**428**	**24**	**148**	470	**24**	151	485	**16**	**99**	501	**16**	100	**482**	**16**	100
xml-17	1,077	44	344	**193**	**15**	**87**	207	**15**	88	**95**	**15**	**23**	105	**15**	24	113	**15**	24
xml-18	792	**39**	**230**	**718**	**39**	231	771	**39**	232	481	**16**	92	**466**	**16**	**90**	488	**16**	**90**
xml-19	**1,150**	56	531	1,185	**54**	508	1,218	**54**	**507**	654	**36**	**166**	**618**	**36**	**166**	679	**36**	**166**
xml-20	2,180	**99**	585	**2,166**	**99**	591	2,215	**99**	**584**	2,546	**64**	385	2,491	**64**	**380**	**2,415**	**64**	**380**
xml-21	2,879	92	813	2,661	**90**	808	**2,594**	**90**	**799**	**1,675**	**46**	**254**	1,792	**46**	265	1,786	**46**	265
xml-22	**1,089**	**61**	**507**	1,193	**61**	510	1,219	**61**	508	343	**24**	94	**327**	**24**	**93**	366	**24**	**93**
xml-23	**3,650**	**112**	**1027**	5,064	165	1434	5,202	165	1434	**2,253**	**48**	**340**	2,702	51	389	2,604	51	389
xml-24	2,084	97	601	**2,002**	**96**	**584**	2,005	**96**	585	2,371	**70**	398	2,418	**70**	**397**	**2,315**	**70**	**397**
xml-25	3,917	135	1125	**2,856**	135	**837**	4,138	**132**	1179	2,474	**55**	**370**	2,516	**55**	372	**2,472**	**55**	371
xml-26	4,687	131	1239	4,321	138	1226	**4,026**	**126**	**1121**	3,150	**64**	**437**	3,200	**64**	440	**3,016**	**64**	439
xml-27	6,456	188	1720	5,816	**186**	1630	**5,802**	**186**	**1597**	5,057	**102**	731	5,018	**102**	**724**	**4,862**	**102**	725
xml-28	5,233	**159**	1537	5,001	**159**	1477	**4,946**	**159**	**1433**	4,229	**97**	**637**	4,298	**97**	638	**4,152**	**97**	638
xml-29	4,741	142	1323	**4,281**	**138**	1239	4,310	**138**	**1231**	2,175	**48**	**338**	2,238	**48**	342	**2,149**	**48**	341
xml-30	4,996	**147**	1365	4,678	**147**	1351	**4,662**	**147**	**1328**	4,981	**78**	710	4,878	**78**	710	**4,799**	**78**	**709**
Mean	1,594	68	521	**1,252**	60	438	1,273	**59**	**437**	**829**	**30**	**156**	837	**30**	157	**829**	**30**	**156**

In Perses, C programs show 4.57% fewer queries and 7.02% less time ($p = 3.05 \times 10^{-5}$), while XML shows negligible impact (0.58% more queries, 0.08% less time, $p > 0.05$).

4.4.3 Analysis. SA_{ProbDD} differs from SA_{DDMIN} in how it operates: while SA_{DDMIN} deterministically partitions inputs using structural heuristics, SA_{ProbDD} probabilistically biases selection using structural weights. This indirect influence smooths query-cost variation, explaining the closer

alignment between query and time savings. For example, in HDD-XML, structural weighting likely promotes early removal of deeply nested constructs, yielding slightly greater time savings despite moderate query reduction.

SA_{ProbDD} also shows better query-to-time ratios than SA_{DDMIN} in Perses-C (↓ 4.57% queries → ↓ 7.02% time vs. ↓ 8.38% → ↓ 3.20%), suggesting that probabilistic learning better leverages limited structural signals. However, when structural complexity is absent—as in Perses-XML—both variants show negligible impact, confirming that structural guidance is effective only when meaningful structure exists.

RQ2: SA_{ProbDD} improves performance in HDD with balanced query-to-time ratios, showing that structural weights integrate well with probabilistic selection. Unlike SA_{DDMIN}, whose gains over DDMIN are larger but more input-dependent (e.g., limited on Perses-XML), SA_{ProbDD} yields smaller yet consistently positive gains over ProbDD, suggesting that probabilistic selection aligns more steadily with structural guidance. In frameworks like Perses, where built-in transformations reduce structural complexity—especially on XML with its simpler, flatter structure—gains remain modest, confirming that structural analysis adds value only when meaningful structure is present.

4.5 RQ3: SADD vs. WDD

RQ3 examines *when* structural weighting matters most by comparing SADD's multidimensional structural signal with WDD's token-based weighting under an otherwise identical weighted framework. We compare SADD's structure-aware weighting with WDD's token-based approach to assess whether our geometric and information-theoretic metrics offer advantages over simple token counting. This isolates the effect of weight sophistication within a shared weighted-partitioning framework. Each technique has two variants: SA_{DDMIN} and SA_{ProbDD} for SADD, and W_{DDMIN} and W_{ProbDD} for WDD. Detailed results are shown in Tables 2 and 3.

4.5.1 Effectiveness. Output size differences between SADD and WDD are minimal. In HDD, SA_{DDMIN} yields 4.96% larger outputs than W_{DDMIN} on C but matches on XML ($p = 0.069$), while SA_{ProbDD} produces 1.55% smaller outputs on both C and XML ($p = 0.343$). In Perses, all differences are below 2% with $p > 0.2$, indicating comparable effectiveness.

4.5.2 Efficiency. Across all 62 benchmarks, SA_{DDMIN} is on average 4.82% faster than W_{DDMIN} and SA_{ProbDD} 1.44% faster than W_{ProbDD} under HDD, while under Perses SA_{ProbDD} is 0.62% faster and SA_{DDMIN} 1.34% slower. This indicates that structural weighting is generally beneficial under HDD but offers limited gains under Perses. In HDD -C, SA_{DDMIN} achieves 15.04% faster reduction and 15.12% fewer queries than W_{DDMIN} ($p = 1.86 \times 10^{-6}$), showing clear gains on structurally diverse programs. In contrast, on HDD -XML, it is 7.43% slower despite nearly identical queries (−0.11%, $p = 0.047$) suggesting overhead outweighs benefits on simpler, flatter structures.

For probabilistic variants, the gap is smaller: SA_{ProbDD} is 4.28% faster on C but 1.68% slower on XML than W_{ProbDD}, suggesting that probabilistic selection aligns more consistently with structural guidance (as seen in Section 4.4).

Under Perses, differences are minimal—SA_{DDMIN} is 2.13% slower on C and 0.50% slower on XML than W_{DDMIN}, while SA_{ProbDD} differs little from W_{ProbDD} (0.30% faster on C and 0.96% faster on XML)—consistent with Sections 4.3 and 4.4.

4.5.3 Analysis. The comparison between SADD and WDD isolates the effect of weight modeling under an otherwise identical control structure. In both deterministic (ddmin-style) and probabilistic (ProbDD-style) variants, the search space, partitioning workflow, candidate exploration order, and termination conditions are unchanged. The only difference lies in the weighting signal: WDD

uses token counts as a one-dimensional syntactic proxy, whereas SADD uses a multidimensional structural measure (V, U, and B) designed to approximate reduction difficulty more directly. Thus, any observed performance differences can be attributed to the quality of the weighting signal rather than to changes in search strategy or algorithmic design. This controlled design ensures that improvements reflect modeling fidelity rather than search-space alteration.

Three factors shape when SADD outperforms WDD:

- **Structural Diversity.** SADD leverages structural heterogeneity. On C programs with deeply nested and diverse constructs (functions, control flow, declarations), our metrics (depth, branching, entropy) distinguish nodes beyond token counts, yielding gains such as 15.12% fewer queries and 15.04% faster reductions on HDD-C. On XML, which is shallow and regular, these metrics converge to uniformity, offering little advantage over token counting.
- **Weight Sensitivity.** In deterministic DDMIN, weights influence every decision, so accuracy directly impacts convergence. In contrast, probabilistic ProbDD adapts during execution, correcting early weight errors. This explains why SA_{ProbDD} and W_{ProbDD} perform similarly even on complex inputs—the benefit of precise weights is amortized by this adaptive learning.
- **Framework Complexity.** As frameworks progress from basic (HDD) to optimized (Perses), built-in strategies (e.g., structural transformations) reduce reliance on weight accuracy, thereby narrowing the performance gap between SADD and WDD.

RQ3: SADD delivers clear gains over WDD on structurally rich inputs like HDD-C, achieving up to 15.04% less time on average while staying competitive elsewhere. Its near-parity in flat domains suggests that token counts suffice when structure is uniform, whereas structure-aware weighting proves beneficial on inputs with complex, heterogeneous constructs.

4.6 RQ4: Component Contribution Analysis

RQ4 analyzes *why* structural weighting improves reduction by examining the individual and joint contributions of SADD's three structural components (V, U, B) through an ablation study within the HDD framework. We vary $\rho_U, \rho_B \in \{0, 1\}$ while fixing $C = 16$ in Equation (1), yielding four configurations: (1) **V-only (baseline)**, (2) $V + U$, (3) $V + B$, and (4) $V + U + B$. Table 4 reports performance improvements over the V-only baseline. We compute the contributions of U and B as (2)–(1) and (3)–(1), respectively, and define their *synergy* as $(4) - (3) - (2)$, representing interaction effects beyond their individual gains.

4.6.1 Component Effects. On C programs, branching complexity ($V + B$) gives the largest individual gain: 5.91% faster ($p < 0.01$) with 2.49% fewer queries, showing B's effectiveness in capturing control-flow complexity. Decision uniformity ($V + U$) adds 0.96% speedup and 0.95% fewer queries by detecting skewed structures missed by V alone. Using both U and B , $V + U + B$ achieves 3.77% faster reductions and 3.57% fewer queries—less than the sum due to overlapping signals—confirming that both metrics capture meaningful structure.

On XML programs, the trend differs but remains informative. $V + B$ slows by 6.23% ($p < 0.05$), while $V + U$ speeds up by 0.33% with negligible query change. Yet $V + U + B$ cuts the slowdown to 0.97% through positive synergy (+4.93%), suggesting U helps target XML subtrees where B fails to add value, revealing complementary effects even in DSLs with simpler, flatter structures.

4.6.2 Analysis. The ablation shows that each component serves a distinct role. **Geometric volume (V)** forms the foundation, capturing most structural complexity through the depth × leaves product and transforming uniform partitioning into structure-aware reduction. **Branching complexity (B)** performs best on structurally heterogeneous code with varied control structures, yielding the largest gains where high-arity decisions (e.g., switch statements, multi-way conditionals) dominate.

Table 4. Component contributions to SA_{DDMIN} in the HDD framework. Values are geometric means of improvements over the V-only baseline. Synergy is computed as Δ(V+U+B) − Δ(V+U) − Δ(V+B) to measure interaction effects. p-values are from two-sided paired Wilcoxon tests: *: $p < 0.05$ and **: $p < 0.01$.

Framework	Variant	C Programs			XML Documents		
		Time↓	Size↓	Query↓	Time↓	Size↓	Query↓
HDD	Baseline (V-only)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
	V+U	0.96%	-3.22%	0.95%	0.33%	0.00%	0.05%
	V+B	5.91%**	0.22%	2.49%	-6.23%*	0.00%	-0.00%
	V+U+B	3.77%	-0.11%	3.57%	-0.97%*	0.00%	-0.02%
	U contribution	+0.96%	-3.22%	+0.95%	+0.33%	+0.00%	+0.05%
	B contribution	+5.91%	+0.22%	+2.49%	-6.23%	+0.00%	-0.00%
	Synergy (U+B)	-3.10%	+2.89%	+0.13%	+4.93%	+0.00%	-0.07%

Decision uniformity (U) acts as a moderator, detecting structural imbalances that guide when other metrics apply, as shown by its positive synergy when combined with B on XML.

Their interactions are revealing: negative synergy on C (−3.10%) suggests overlapping coverage of complex structures, while positive synergy on XML (+4.93%) indicates complementary specialization. These patterns support our multi-metric design: no single metric captures all structural aspects, and their combination, though occasionally overlapping, enhances robustness across diverse inputs.

Our fixed parameters ($\rho_U = \rho_B = 1$) work well across inputs, but the observed interactions (negative on C but positive on XML) suggest potential for adaptive weighting. Although our current scheme is not fully optimal, lightweight input profiling could tune these weights, boosting gains on suitable inputs while avoiding the 7.43% overhead observed on regular structures like HDD-XML.

4.6.3 Practical Implications. For practitioners, we recommend using V as the core metric—it yields substantial gains with minimal complexity. Adding B benefits structurally diverse programs with complex control flow common in general-purpose languages such as C, while U enhances robustness by moderating other metrics on inputs with mixed structural balance.

Our ablation shows that the relative contribution of each structural component varies across input domains. Nevertheless, the fixed configuration (ρ_U=ρ_B=1, C=16) used throughout our evaluation serves as a robust default that performs competitively across subjects without per-benchmark tuning. The results also confirm that this configuration is not universally optimal: components contribute differently across input types (e.g., the branching-related component is more informative for C than for XML in our study). This suggests a practical path for further improvement—leveraging domain knowledge to pre-tune domain-specific defaults on a small representative sample, so that SADD starts from well-calibrated parameters rather than a single global setting.

RQ4: Geometric volume (V) forms the core of SADD's benefits, with B contributing a 5.91% speedup on structurally diverse code and U enabling positive component interactions. The contrasting synergies—negative on C but positive on XML—support our multi-metric design, showing that different structural patterns benefit from different component combinations.

These component-level findings reinforce our central premise: structural signals provide complementary guidance that token-based proxies cannot uniformly capture.

5 Discussion

The effectiveness of structural weighting depends on two main factors—input heterogeneity and framework architecture—offering practical guidance for researchers and practitioners.

- **When Structural Heterogeneity Matters.** As shown in our comparison of SADD and WDD (Section 4.5), structural analysis benefits heterogeneous inputs but not regular ones. C programs contain diverse constructs (e.g., functions, control flow, and declarations) that WDD's token counting conflates, whereas XML's uniform nesting needs no such distinction. This suggests a guideline: use structural weighting for structurally rich languages (e.g., C/C++ and Rust) and token counting for regular formats (e.g., JSON and CSV).
- **When Framework Architecture Matters.** SADD's impact—57.12% speedup in HDD-C versus 3.20% in Perses-C (SA_{DDMIN} vs. ddmin)—reflects framework design rather than algorithm quality. When ddmin is central (HDD), better partitioning yields large gains; when it is peripheral after built-in structural transformations (Perses), gains are inherently limited. Recognizing these bounds can guide future optimization efforts.

6 Related Work

We position our work relative to two main lines of research: foundational delta debugging algorithms that established systematic reduction through uniform partitioning, and advanced heuristic-based strategies that refine this foundation with non-uniform guidance.

6.1 Foundational Delta Debugging

The ddmin algorithm [34] introduced systematic test-case reduction via iterative subset testing. Its uniform partitioning strategy treats all elements as equally significant, offering generality but often proving inefficient on inputs with heterogeneous structural complexity.

To handle structured data, HDD [15] and its variants [8, 9] apply delta debugging hierarchically across parse tree levels, while Perses [21] leverages formal grammars to guide reductions while preserving syntactic validity.

HDDr [10] replaces HDD's level-wise reduction with a recursive scheme that applies ddmin to the children of each node (i.e., sibling groups) rather than to all nodes at a given depth level, thereby reducing the average size of element lists processed by ddmin. SADD is orthogonal to such traversal-level optimizations: it does not alter the hierarchical scheduling strategy but refines the weighting signal used within each reduction step, and could in principle be composed with HDDr's recursive traversal.

While HDD-style reducers adapt delta debugging to tree-structured inputs through hierarchical scheduling, nodes within the current reduction scope (a depth level in HDD or a sibling group in HDDr) are still treated uniformly during partitioning. SADD complements these approaches by modeling structural complexity differences among nodes within that scope, enabling non-uniform guidance inside each reduction call. This design distinction targets inefficiencies that arise when structural complexity varies significantly within partitions.

6.2 Non-Uniform Reduction Strategies

The limitations of uniform partitioning have driven the development of strategies that guide reduction using element-specific characteristics.

Syntactic Weighting. WDD [39] is the first systematic approach to non-uniform partitioning, using token counts as proxies for element complexity. This demonstrates that incorporating element characteristics can improve reduction efficiency. However, token counts capture only surface-level size—elements of similar syntactic size may differ substantially in structural complexity. SADD refines this idea by replacing token-based proxies with a multidimensional structural measure grounded in geometric and information-theoretic properties of the syntax tree.

Probabilistic Guidance. ProbDD [24] prioritizes candidate subsets using a probabilistic model learned from test outcomes. T-PDD [25] extends ProbDD to tree-structured inputs by constructing

a Bayesian network from the AST, where parent–child dependencies define conditional probability tables capturing syntactic relationships among nodes. At each step, T-PDD selects subsets that maximize expected gain under this dependency-aware model and updates the network after each test. This model-level dependency reasoning is complementary to SADD's weight-level structural priors: T-PDD improves the probabilistic model by relaxing ProbDD's independence assumption, whereas $\text{SA}_{\text{ProbDD}}$ preserves ProbDD's update mechanism and modifies only the gain objective via $w_{\text{struct}}(\cdot)$. These directions operate at different abstraction levels and could be composed. Since our contribution targets weighting within ddmin-style reducers rather than traversal scheduling or probabilistic model design, we use ddmin, ProbDD and WDD as primary experimental baselines.

CDD [37] showed that ProbDD's model effectively behaves as a counter system, confirming that such approaches guide exploration via empirical feedback. CHISEL [7] applies reinforcement learning to identify promising reduction sequences. These element-focused methods ask: "Which elements have worked before?" By contrast, PMA [22] estimates search-space monotonicity rather than element importance, excluding likely non-failure-inducing inputs to avoid redundant tests. SADD addresses a complementary question: "Which elements are structurally complex and thus harder to eliminate?" This enables SADD to provide informed *a priori* partitioning that complements both probabilistic element-focused approaches and search-space-oriented techniques such as PMA.

Transformation-based Enhancement. C-Reduce [18] augments deletion with C/C++-specific transformations such as function inlining and constant propagation. ddSMT [16] and ddSMT2.0 [11] apply SMT-LIBv2-specific transformations (e.g., command-level scoping and term substitution), with ddSMT2.0 adding hybrid strategies that combine transformations with hierarchical reduction. Vulcan [29] integrates transformations with ddmin-based methods using techniques such as Identifier Replacement and Tree-Based Local Exhaustive Enumeration. While these systems improve reduction effectiveness, they may incur efficiency costs due to large transformation search spaces. SADD contributes orthogonally by improving the efficiency of the core partitioning and ranking mechanisms on which these frameworks rely.

7 Conclusion

We presented Structure-Aware Delta Debugging (SADD), which enhances delta debugging by incorporating geometric and information-theoretic analyses of program structure. Our key insight is that structural properties—geometric volume, decision uniformity, and branching complexity—capture reduction complexity more accurately than token counts, particularly for structurally heterogeneous programs. We instantiated SADD in two algorithms, SA_{ddmin} and $\text{SA}_{\text{ProbDD}}$, extending both deterministic and probabilistic reduction strategies.

Evaluation on 62 real-world benchmarks shows that SADD delivers substantial speedups when structure is rich and ddmin is central, while remaining competitive elsewhere. An ablation study confirms that geometric volume forms the foundation, with decision uniformity and branching complexity providing complementary refinements.

Crucially, our results delineate when structural analysis adds value: on diverse, tree-structured inputs within delta-debugging-centric frameworks. This guidance can help practitioners choose effective reduction strategies and informs future work on context-aware minimization. Our evaluation focuses on C and XML inputs; while these represent structurally heterogeneous domains, results may differ for languages with substantially different AST characteristics (e.g., indentation-sensitive or macro-heavy languages), which we leave for future investigation. SADD demonstrates that modeling structural complexity can substantially advance automated test case reduction.

8 Data Availability

All data, source code, and artifacts are available at https://anonymous.4open.science/r/SADD-5383/.

References

[1] BaseX Team. 2025. BaseX: XML Database and XQuery Processor. https://github.com/BaseXdb/basex. [Online; accessed 01-September-2025].

[2] GCC Bugzilla. 2015. Bug 71626. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=71626. [Online; accessed 01-September-2025].

[3] Junjie Chen, Guancheng Wang, Dan Hao, Yingfei Xiong, Hongyu Zhang, and Lu Zhang. 2020. History-guided configuration diversification for compiler test-program generation. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE '19)*. IEEE Press, San Diego, California, 305–316. doi:10.1109/ASE.2019.00037

[4] Holger Cleve and Andreas Zeller. 2005. Locating causes of program failures. In *Proceedings of the 27th International Conference on Software Engineering* (St. Louis, MO, USA) *(ICSE '05)*. Association for Computing Machinery, New York, NY, USA, 342–351. doi:10.1145/1062455.1062522

[5] Alastair F. Donaldson, Paul Thomson, Vasyl Teliman, Stefano Milizia, André Perez Maselco, and Antoni Karpiński. 2021. Test-case reduction and deduplication almost for free with transformation-based compiler testing. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) *(PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 1017–1032. doi:10.1145/3453483.3454092

[6] GCC Wiki. 2025. A Guide to Testcase Reduction. https://gcc.gnu.org/wiki/A_guide_to_testcase_reduction. Online; accessed 01-September-2025.

[7] Kihong Heo, Woosuk Lee, Pardis Pashakhanloo, and Mayur Naik. 2018. Effective Program Debloating via Reinforcement Learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) *(CCS '18)*. Association for Computing Machinery, New York, NY, USA, 380–394. doi:10.1145/3243734.3243838

[8] Renáta Hodován and Ákos Kiss. 2016. Modernizing hierarchical delta debugging. In *Proceedings of the 7th International Workshop on Automating Test Case Design, Selection, and Evaluation* (Seattle, WA, USA) *(A-TEST 2016)*. Association for Computing Machinery, New York, NY, USA, 31–37. doi:10.1145/2994291.2994296

[9] Renáta Hodován, Ákos Kiss, and Tibor Gyimóthy. 2017. Coarse Hierarchical Delta Debugging. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE Computer Society, Shanghai, China, 194–203. doi:10.1109/ICSME.2017.26

[10] Ákos Kiss, Renáta Hodován, and Tibor Gyimóthy. 2018. HDDr: a recursive variant of the hierarchical Delta debugging algorithm. In *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation* (Lake Buena Vista, FL, USA) *(A-TEST 2018)*. Association for Computing Machinery, New York, NY, USA, 16–22. doi:10.1145/3278186.3278189

[11] Gereon Kremer, Aina Niemetz, and Mathias Preiner. 2021. ddSMT 2.0: Better Delta Debugging for the SMT-LIBv2 Language and Friends. In *Computer Aided Verification*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer International Publishing, Cham, 231–242.

[12] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. *SIGPLAN Not.* 49, 6 (jun 2014), 216–226. doi:10.1145/2666356.2594334

[13] LLVM. 2025. How to Submit an LLVM Bug Report. https://llvm.org/docs/HowToSubmitABug.html. Online; 01-September-2025.

[14] Christopher Manning and Hinrich Schutze. 1999. *Foundations of statistical natural language processing*. MIT press.

[15] Ghassan Misherghi and Zhendong Su. 2006. HDD: hierarchical delta debugging. In *Proceedings of the 28th International Conference on Software Engineering* (Shanghai, China) *(ICSE '06)*. Association for Computing Machinery, New York, NY, USA, 142–151. doi:10.1145/1134285.1134307

[16] Aina Niemetz and Armin Biere. 2013. ddSMT: a delta debugger for the SMT-LIB v2 format. In *Proceedings of the 11th International Workshop on Satisfiability Modulo Theories, SMT*. 8–9.

[17] PLUverse. 2025. perses: Language and systems support for typedirected program transformations. https://github.com/uw-pluverse/perses. Online; Accessed: 01-September-2025.

[18] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) *(PLDI '12)*. Association for Computing Machinery, New York, NY, USA, 335–346. doi:10.1145/2254064.2254104

[19] C. E. Shannon. 1948. A mathematical theory of communication. *The Bell System Technical Journal* 27, 3 (1948), 379–423. doi:10.1002/j.1538-7305.1948.tb01338.x

[20] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding compiler bugs via live code mutation. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Amsterdam, Netherlands) *(OOPSLA 2016)*. Association for Computing Machinery, New York, NY, USA, 849–863. doi:10.1145/2983990.2984038

[21] Chengnian Sun, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. 2018. Perses: syntax-guided program reduction. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) *(ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 361–371. doi:10.1145/3180155.3180236

[22] Yonggang Tao and Jingling Xue. 2025. Accelerating Delta Debugging through Probabilistic Monotonicity Assessment. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25)*. Association for Computing Machinery, New York, NY, USA, 12–22. doi:10.1145/3756681.3756940

[23] Yongqiang Tian, Xueyan Zhang, Yiwen Dong, Zhenyang Xu, Mengxiao Zhang, Yu Jiang, Shing-Chi Cheung, and Chengnian Sun. 2023. On the Caching Schemes to Speed Up Program Reduction. *ACM Transactions on Software Engineering and Methodology* 33, 1 (2023), 1–30.

[24] Guancheng Wang, Ruobing Shen, Junjie Chen, Yingfei Xiong, and Lu Zhang. 2021. Probabilistic Delta debugging. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 881–892. doi:10.1145/3468264.3468625

[25] Guancheng Wang, Yiqian Wu, Qihao Zhu, Yingfei Xiong, Xin Zhang, and Lu Zhang. 2023. A Probabilistic Delta Debugging Approach for Abstract Syntax Trees. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE Press, Florence, Italy, 763–773. doi:10.1109/ISSRE59848.2023.00060

[26] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. Validating SMT solvers via semantic fusion. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) *(PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 718–730. doi:10.1145/3385412.3385985

[27] R. F. Woolson. 2005. Wilcoxon Signed-Rank Test. In *Encyclopedia of Biostatistics*. John Wiley & Sons, Ltd. doi:10.1002/0470011815.b2a15177

[28] Zhenyang Xu, Yongqiang Tian, Mengxiao Zhang, Jiarui Zhang, Puzhuo Liu, Yu Jiang, and Chengnian Sun. 2024. T-Rec: Fine-Grained Language-Agnostic Program Reduction Guided by Lexical Syntax. *ACM Transactions on Software Engineering and Methodology* (2024).

[29] Zhenyang Xu, Yongqiang Tian, Mengxiao Zhang, Gaosen Zhao, Yu Jiang, and Chengnian Sun. 2023. Pushing the Limit of 1-Minimality of Language-Agnostic Program Reduction. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 97 (apr 2023), 29 pages. doi:10.1145/3586049

[30] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 283–294. doi:10.1145/1993498.1993532

[31] Peisen Yao, Heqing Huang, Wensheng Tang, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2021. Skeletal approximation enumeration for SMT solver testing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1141–1153. doi:10.1145/3468264.3468540

[32] Andreas Zeller. 1999. Yesterday, my program worked. Today, it does not. Why? *ACM SIGSOFT Software engineering notes* 24, 6 (1999), 253–267.

[33] Andreas Zeller. 2002. Isolating cause-effect chains from computer programs. In *Proceedings of the 10th ACM SIGSOFT Symposium on Foundations of Software Engineering* (Charleston, South Carolina, USA) *(SIGSOFT '02/FSE-10)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/587051.587053

[34] A. Zeller and R. Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200. doi:10.1109/32.988498

[35] Haotian Zhang, Mengfei Ren, Yu Lei, and Jiang Ming. 2022. One size does not fit all: security hardening of MIPS embedded systems via static binary debloating for shared libraries. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 255–270. doi:10.1145/3503222.3507768

[36] Mengxiao Zhang, Yongqiang Tian, Zhenyang Xu, Yiwen Dong, Shin Hwei Tan, and Chengnian Sun. 2024. LPR: Large Language Models-Aided Program Reduction. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 261–273.

[37] Mengxiao Zhang, Zhenyang Xu, Yongqiang Tian, Xinru Cheng, and Chengnian Sun. 2025. Toward a Better Understanding of Probabilistic Delta Debugging. (2025), 2024–2035. doi:10.1109/ICSE55347.2025.00117

[38] Mengxiao Zhang, Zhenyang Xu, Yongqiang Tian, Yu Jiang, and Chengnian Sun. 2023. PPR: Pairwise Program Reduction. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 338–349.

[39] Xintong Zhou, Zhenyang Xu, Mengxiao Zhang, Yongqiang Tian, and Chengnian Sun. 2025. WDD: Weighted Delta Debugging. (2025). doi:10.1109/ICSE55347.2025.00071